

RAG Advanced

Warum naive Retrieval-Pipelines scheitern und wie Hybrid Search, Reranking und Evaluation sie retten.



Use Case: NordLicht Logistik GmbH internes Wissen für Kundenservice & Personal

Was euch heute erwartet



Teil 1 · RAG-Grundlagen

Embeddings, Vektor-DBs, Chunking, der Prompt und warum naives RAG scheitert.



Teil 2 · Advanced RAG

Hybrid Search, RRF, Reranking, die Metadaten-Falle und Evaluation.



Teil 3 · Über RAG hinaus

Fine-Tuning vs. RAG, LoRA/QLoRA und ein Ausblick auf Agents.

Format



Mitmach-Workshop

4 Notebooks in Colab, kein Frontalvortrag



~ 2 Stunden

Konzept · Hands-on · Reflexion



In 2er-Teams

am eigenen Rechner, mit Betreuung



Komplett kostenlos

Open-Source-Modelle, keine API-Keys

Was ihr danach könnt



Eine RAG-Pipeline erklären

Von Chunking über Embeddings und Vektorsuche bis zur Antwortgenerierung.



Verstehen, warum naives RAG scheitert

Vocabulary Mismatch, fehlende exakte Begriffe, veraltete Quellen.



Advanced-RAG einordnen

Hybrid Search, Reciprocal Rank Fusion und Cross-Encoder-Reranking gezielt einsetzen.



Retrieval-Qualität messen

Mit Hit@3, MRR und der RAGAS-Logik eine Pipeline objektiv bewerten.



RAG von Fine-Tuning abgrenzen

Wissensproblem vs. Verhaltensproblem und wann welcher Hebel greift.



Agents einordnen

Wann ein einzelner Retrieval-Schritt nicht reicht und ReAct ins Spiel kommt.

TEIL 1

RAG-Grundlagen

Wie eine Retrieval-Pipeline von Grund auf funktioniert

- Das Wissensproblem der LLMs und die Idee von RAG
- Embeddings, Bi-Encoder und Vektordatenbanken
- Chunking-Strategien und der erweiterte Prompt
- Warum die naive Pipeline an ihre Grenzen stößt



LLMs haben ein Wissensproblem



Statisches Wissen

Das Modell kennt nur seinen Trainingsstand. Interne Dokumente und aktuelle Fakten fehlen.



Keine Quellen

Antworten sind plausibel formuliert, aber nicht belegt, Halluzinationen sind strukturell angelegt.



Kein Zugriff

Firmenwissen liegt in Systemen, auf die das LLM von sich aus keinen Zugriff hat.



Die Lösung zur Laufzeit: relevante Dokumente suchen und dem LLM als Kontext mitgeben. Es antwortet aus dem, was es gerade liest, statt aus dem Gedächtnis. Das ist **Retrieval-Augmented Generation**.

Wenn der Bot Unsinn erzählt, haftet das Unternehmen



Der Fall Air Canada (2024)

- Ein Support-Chatbot erfindet eine Rückerstattungs-Regel, die es nie gab.
- Ein Kunde verlässt sich darauf und klagt.
- Das Gericht entscheidet: Die Airline haftet für die Aussage ihres Bots.

Die Lektion

Eine flüssige, falsche Antwort ist teurer als gar keine.

Vertrauen, Recht und Reputation hängen an der Faktentreue.

RAG in zwei Phasen

PHASE 1 · OFFLINE (einmalig)



Chunking

Dokumente in Häppchen teilen



Embedding

Jeder Chunk wird zum Vektor



Indexieren

Vektoren in die Vektor-Datenbank

PHASE 2 · ZUR LAUFZEIT (pro Frage)



Retrieval

Passende Chunks zur Frage suchen



Augmentierung

Chunks in den Prompt einbetten



Generierung

LLM antwortet aus dem Kontext

Der Schlüsselsatz für heute: Die Generierung kann nur so gut sein wie das Retrieval.

Was ist ein Embedding?

Vom Wort zur Zahl

Ein neuronales Netz rechnet mit Vektoren, ein Wort wie „Hamburg“ ist keine Zahl. Ein Embedding bildet Text auf einen dichten Vektor ab (typisch 128–4096 Dimensionen).

Die zentrale Eigenschaft

Ähnliche Bedeutung → ähnliche Vektoren. „Paket“ und „Sendung“ landen nah beieinander, „Paket“ und „Urlaub“ weit auseinander. Das macht Bedeutung rechenbar.

Nähe messen: Kosinus-Ähnlichkeit

Der Winkel zwischen zwei Vektoren, nicht ihre Länge, misst, wie ähnlich zwei Texte sind:

$$\cos(v_q, v_d) \in [-1, 1]$$

1,0

0,5

0

identisch

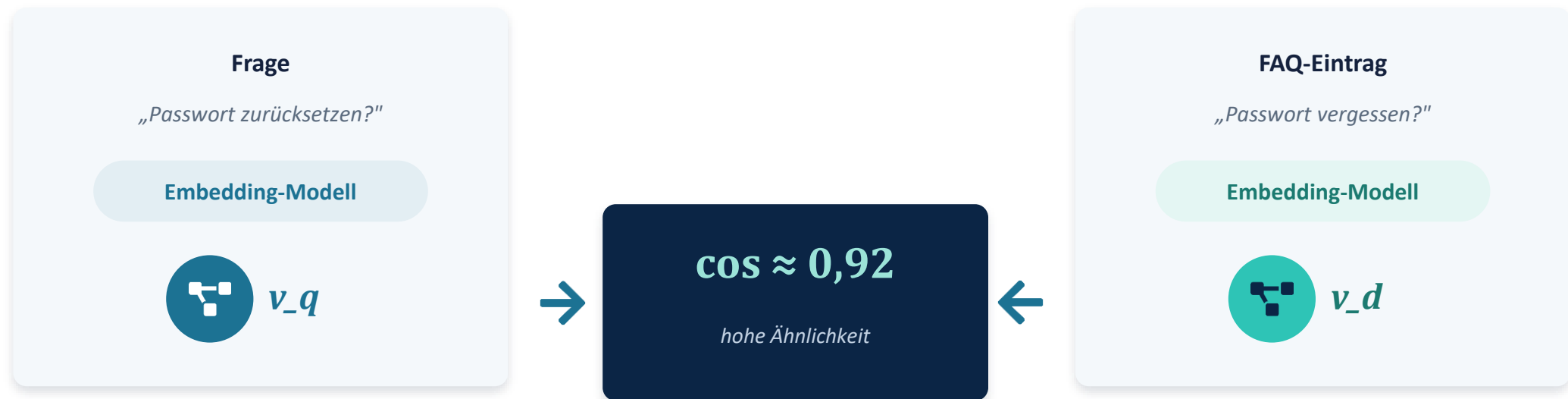
verwandt

unabhängig

Beispiel: „Passwort zurücksetzen?“ ↔ „Passwort vergessen?“
→ $\cos \approx 0,92$

Der Bi-Encoder (SBERT)

Zwei Encoder teilen sich die Gewichte und verarbeiten Frage und Dokument unabhängig voneinander, dadurch lassen sich alle Dokument-Vektoren einmal vorberechnen.



Warum das zählt: Dokument-Vektoren werden einmal offline berechnet. Zur Laufzeit muss nur die Frage encodiert und der Abstand verglichen werden → aus Stunden werden Sekunden.

Warum eine Vektordatenbank?



Naiv: Brute-Force

Bei jeder Frage den Abstand zu allen Vektoren rechnen. Bei 1 Mio. Vektoren $\times$ 768 Dim. sind das 750 Mio. Operationen pro Query.

Komplexität $O(n)$



Lösung: ANN

Approximate Nearest Neighbor: nicht exakt den nächsten, sondern schnell einen sehr guten Nachbarn finden. In der Praxis vernachlässigbarer Qualitätsverlust.

Komplexität $O(\log n)$

HNSW · Hierarchical Navigable Small World

(Malkov & Yashunin, 2018)

Ein mehrschichtiger Graph: Die Suche startet oben mit großen Sprüngen, navigiert zur richtigen Region und findet auf der untersten Ebene das genaue Ergebnis.



Ebene 2 $\rightarrow$ 0

Welche Vektordatenbank?



pgvector

PostgreSQL-Extension

Vektorsuche direkt in Postgres,
kein neuer Stack. Ideal für
kleinere Projekte.

★ unser Stack-Vorbild



Qdrant

Dedicated · Rust

Sehr schnell, speichereffizient,
Hybrid Search nativ. Gut für
Production.



Chroma

Lightweight

Einfachster Einstieg, In-Process-
Python. Für Notebooks und
Prototypen.



Weaviate

Cloud-native

Hybrid & multimodal eingebaut,
GraphQL-API für komplexe
Schemata.

Faustregel: pgvector, wenn es einfach bleiben soll. Qdrant, wenn Hybrid Search und Performance gefragt sind. Im Workshop nutzen wir bewusst nur numpy: 36 Chunks brauchen keinen Index.

Chunking-Strategien

Embedding-Modelle verarbeiten begrenzte Textlängen, und kleinere Einheiten lassen sich präziser wiederfinden. Wie man teilt, entscheidet über die Retrieval-Qualität.



Fixed-Size

Der robuste Default

- Feste Tokenzahl je Chunk
- Overlap gegen Kontextverlust
- Sweet Spot: 256 - 512 Token
- Overlap: 50 - 100 Token (10–20 %)



Semantic

Thematisch kohärent

- Kosinus zwischen Nachbarsätzen
- Teilen bei Ähnlichkeitsabfall
- Kohärente Einheiten
- Langsamer, variable Größen



Hierarchical

State of the Art

- Kleine Child-Chunks fürs Retrieval
- Große Parent-Chunks für den LLM-Kontext
- Karteikarte suchen, Kapitel lesen
- Für komplexe Dokumente

In der Praxis: Fixed-Size als Baseline. Semantic oder Hierarchical, wenn die RAGAS-Scores niedrig sind. Chunk-Größen immer empirisch testen. Im Workshop: 1 Absatz = 1 Chunk.

Der erweiterte Prompt

1

System-Instruktion

„Beantworte die Frage ausschließlich anhand des Kontexts. Steht die Antwort nicht drin, sage es ehrlich.“

2

Kontext-Chunks

Die k ähnlichsten Dokumentabschnitte, nummeriert und mit Quellenangabe.

3

Nutzeranfrage

Die ursprüngliche Frage, unverändert angehängt.



Der Effekt

Das LLM wird gezwungen, auf echten Quellen zu basieren. Halluzinationen sind nicht ausgeschlossen, aber die Fehlerquote sinkt erheblich.



DSGVO-Vorteil

Nicht-parametrisches Wissen ist löschar:
Dokument aus dem Index entfernen fertig. Kein
Modell-Retraining nötig.

Genau diesen Prompt zeigt Notebook 01 → die „Generierung“ simulieren wir dann deterministisch, damit alles kostenlos bleibt.

Die naive Pipeline → unser Startpunkt



Eure Übung 1: Kosinus-Ähnlichkeit und die Top-k-Suche selbst implementieren. Eine -Selbsttest-Zelle prüft euch automatisch.

Zwei blinde Flecken der Vektorsuche



Vocabulary Mismatch

„SLA für P1-Tickets?“

Das Wiki schreibt „Service Level Agreement“ und „Priority-1-Vorfälle“, aber nie die Kürzel. Bei internen Codes und Versionsnummern bleibt die Bedeutungssuche unzuverlässig.



Das andere Extrem: BM25

„Kosten für Cloud-Speicher“

Reine Keyword-Suche findet keine Seite, die nur von „Preisen für Objektspeicherung“ spricht. Sie kennt exakte Wörter, aber keine Bedeutung.

Die Diagnose: Ein Verfahren allein reicht nicht. Wir brauchen **Bedeutung UND exakte Begriffe** → führt uns zu Hybrid Search.



JETZT IHR · HANDS-ON

Notebook 01

Naive RAG bauen & an die Grenze bringen

- Chunking, Embeddings und die Vektorsuche zum Laufen bringen
- Übung 1: Kosinus-Ähnlichkeit + Top-k selbst implementieren
- Die zwei blinden Flecken am eigenen Rechner erleben

TEIL 2

Advanced RAG

Von brauchbaren Treffern zu präzisen, überprüfbaren Antworten

- Hybrid Search: BM25 und Vektorsuche vereint (RRF)
- Cross-Encoder-Reranking und das zweistufige Muster
- Die Metadaten-Falle: Relevanz ist nicht Gültigkeit
- Evaluation mit Hit@3, MRR und der RAGAS-Logik



Hybrid Search: zwei Sucher, zwei Stärken



BM25 · Keyword-Suche

- Vergleicht exakte Wortformen (seit den 90ern Standard).
- Stark bei Codes, Produktnamen, Fachbegriffen.
- Blind für Synonyme und Umschreibungen.



Dense · Vektorsuche

- Vergleicht Bedeutung über Embeddings.
- Stark bei Synonymen und Umgangssprache.
- Schwach bei exakten, semantisch armen Begriffen.



Idee: beide parallel laufen lassen und die Ergebnislisten fair verschmelzen. Aber die Scores sind **nicht vergleichbar** (BM25: 0...>10, Kosinus: -1...1). Wie fusioniert man das?

BM25 verstehen: drei Zutaten

BM25 („Best Matching 25“) ist der Standard klassischer Suchmaschinen. Ein Dokument ist relevant, wenn es die Suchbegriffe passend enthält, drei Faktoren zusammen:



Term Frequency

Wie oft steht der Begriff im Dokument?
Öfter = relevanter, aber mit abnehmendem Grenznutzen.



Inverse Document Frequency

Wie selten ist der Begriff im ganzen Korpus?
Seltene Wörter tragen mehr Information als „und“ oder „die“.



Längen-Normalisierung

Lange Dokumente enthalten Begriffe naturgemäß öfter. BM25 rechnet das heraus, um kurze nicht zu benachteiligen.

Warum es die Vektorsuche ergänzt: BM25 matcht exakte Zeichenketten. Genau deshalb trifft es Fehlercodes wie „NL-410“ sicher dort, wo Embeddings scheitern. Es versteht nur keine Synonyme. Die perfekte Ergänzung zur Bedeutungssuche.

Reciprocal Rank Fusion (RRF)

Nur Ränge zählen - keine rohen Scores.

$$RRF(d) = \sum 1 / (k + rang_r(d))$$

k = 60 · robuster Default aus dem Original-Paper (Cormack et al., SIGIR 2009).

Eure Übung 2: RRF implementieren. Der Selbsttest ist genau dieses Beispiel → B muss gewinnen (1/62 + 1/61).

Worked Example (aus VL 3)

Dok.	BM25	Dense	RRF	
A	1	3	0,0321	
B	2	1	0,0326	★
C	3	2	0,0319	

B gewinnt: Rang 2 und Rang 1. Konsistenz über beide Listen wird belohnt — nicht der eine Spitzenplatz.



JETZT IHR · HANDS-ON

Notebook 02

Hybrid Search & RRF selbst bauen

- BM25 gegen die Vektorsuche antreten lassen (das Duell)
- Übung 2: Reciprocal Rank Fusion implementieren
- Beide blinden Flecken aus NB 01 verschwinden ohne neue Nachteile

Cross-Encoder Reranking

Bi-Encoder (Retrieval)

Frage und Dokument getrennt encodiert. Vektoren sind vorberechenbar.

Schnell · Millionen Chunks

Ungenauer im Ranking

Query und Dokument „sehen“ einander nie.

Cross-Encoder (Reranking)

Liest [Frage] [SEP] [Dokument] gemeinsam und gibt einen direkten Relevanz-Score.

Sehr präzise

Teuer · nur für Top-N

Volle Aufmerksamkeit über das Frage-Dokument-Paar.



Eure Übung 3: das Reranking implementieren, den Cross-Encoder alle Kandidaten-Paare bewerten lassen und die Top-N zurückgeben.

Das zweistufige Muster

Der Trick: erst breit sammeln (billig), dann präzise sortieren (teuer, aber nur für wenige Kandidaten).



Stage 1: Recall

Billige Verfahren holen viele Kandidaten. Lieber ein paar zu viel als das Richtige verpassen.



Stage 2: Precision

Der teure Cross-Encoder bewertet nur die 50 Kandidaten und hebt die besten 5 ins LLM.



JETZT IHR · HANDS-ON

Notebook 03

Reranking bauen und dem System eine Falle stellen

- Übung 3: Cross-Encoder-Reranking implementieren
- Die voll ausgebaute Pipeline an einer Alltagsfrage testen
- Übung 4: den Fehler beheben, sobald wir ihn verstanden haben

Relevanz ist nicht Gültigkeit



„Wie viele Tage Homeoffice sind erlaubt?“

Die Pipeline antwortet:

„maximal zwei Tage“

Quelle: **Homeoffice-Regelung (2022)** · Status: **archiviert**

*Flüssig. Mit Quellenangabe. Völlig überzeugend und falsch.
Gültig sind seit 2025 drei Tage.*

Warum passiert das?

BM25

liebt 2022, dort steht wörtlich & oft „Homeoffice“. 2025 spricht von „mobilem Arbeiten“.

Dense

findet 2022 ebenfalls hochrelevant → es geht semantisch exakt um die Frage.

Cross-Encoder

bewertet literale Relevanz. Der 2022-Text passt perfekt, dass er nicht mehr gilt, steht nicht drin.

Alle drei taten exakt, wofür sie gebaut sind. Retrieval-Qualität ≠ korrekte Antwort.

Der Fix: Metadaten ernst nehmen

Ein Filter vor dem Reranking

```
kandidaten = [c for c in kandidaten  
                if c["status"] != "archiviert"]
```

Die Chunks trugen status und jahr die ganze Zeit mit sich.

Jetzt antwortet die Pipeline:

„drei Tage“

Quelle: **Richtlinie Mobiles Arbeiten (2025)** · gültig ✓

Der Selbsttest bestätigt: archivierte Chunks sind raus, die gültige Quelle gewinnt.



Der Code war eine Zeile, die Lektion ist es nicht

- Gültigkeit, Aktualität, Zuständigkeit stehen in Metadaten, nicht im Text und keine Retrieval-Stufe sieht sie von allein.
- Der echte Engpass in Unternehmen: Wer pflegt den Status? Was tun bei widersprüchlichen, nicht archivierten Dokumenten?

Beweisen statt behaupten: die Evaluation

Zwei Metriken auf 15 Gold-Fragen



Hit@3

Ist ein relevantes Dokument in den Top-3?

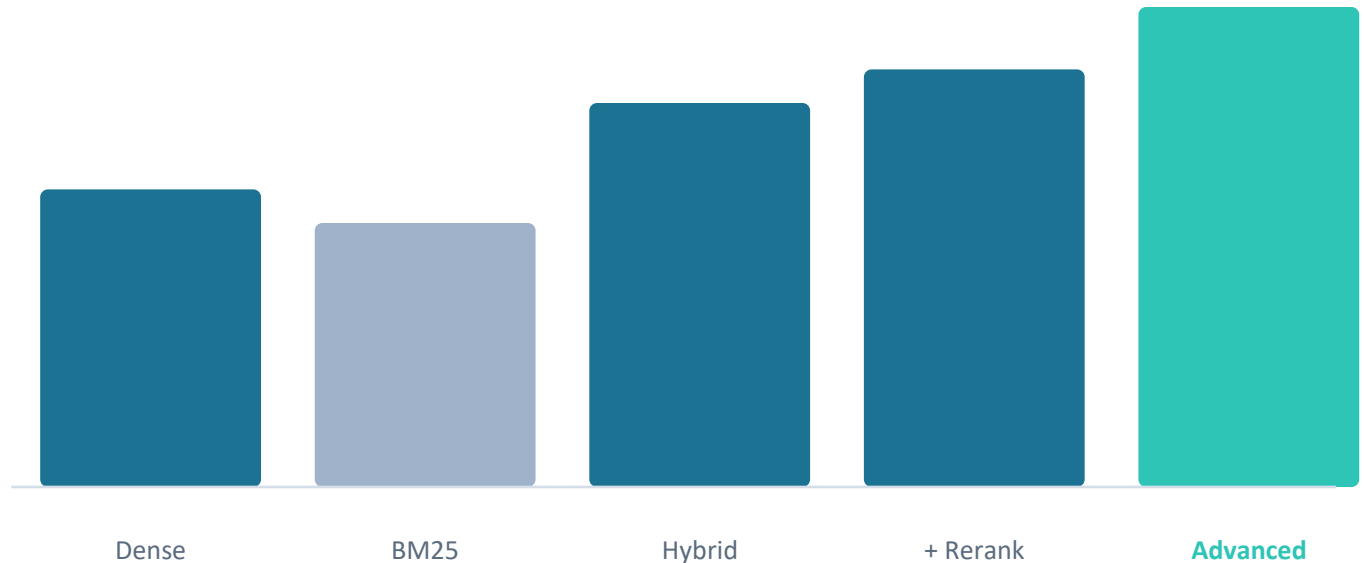


MRR

Wie weit oben steht der erste Treffer?

Rang 1 → 1,0 · Rang 2 → 0,5 · Rang 3 → 0,33

Retrieval-Qualität je Ausbaustufe



Precision vs. Recall



Recall

Von allem Relevanten — wie viel habe ich gefunden? Hohes k, viele Kandidaten: nichts wird übersehen, aber es kommt auch Ballast mit.



Precision

Von dem, was ich gefunden habe — wie viel ist wirklich relevant? Reranking und kleines k halten den Kontext sauber, riskieren aber, Gutes wegzulassen.



So löst unsere Pipeline den Kompromiss auf

Stage 1 (Hybrid Search) maximiert **Recall** — viele Kandidaten, nichts verpassen. **Stage 2 (Reranking)** maximiert **Precision** — nur das Beste geht ans LLM. Genau das messen Hit@3 (Recall-nah) und MRR (Precision-nah).

RAGAS: vier Metriken

Unsere Metriken bewerten nur das Retrieval. RAGAS misst zusätzlich die Generierung, braucht dafür aber ein LLM als Judge (= API-Kosten).

Retrieval

Context Recall

Ist die nötige Information überhaupt im abgerufenen Kontext?

≈ unser Hit@3

Retrieval

Context Precision

Steht das Relevante oben, ohne viel Irrelevantes dazwischen?

≈ unser MRR

Generierung

Faithfulness

Hält sich die Antwort an den Kontext, ohne zu erfinden?

braucht LLM-Judge

Generierung

Answer Relevancy

Beantwortet die Antwort tatsächlich die gestellte Frage?

braucht LLM-Judge

Diagnose-Logik: wo klemmt es?

Der eigentliche Wert von RAGAS: Die Kombination der Werte sagt nicht nur, DASS etwas falsch ist, sondern WO und was zu tun ist.

↓ Faithfulness, ↑ Context Precision

Das LLM ignoriert den Kontext oder überschreibt ihn mit eigenem Wissen.



System-Prompt schärfen, Modell wechseln

↓ Context Recall, ↑ Faithfulness

Der Retriever findet nicht alles, nutzt aber gut, was er findet.



Chunking überdenken, Hybrid Search, k erhöhen

↓ Context Precision, ↑ Context Recall

Der Retriever findet viel, aber auch viel Irrelevantes.



Reranking einsetzen, k reduzieren

↓ Answer Relevancy

Die Antwort weicht vom Thema ab.



Prompt Engineering



JETZT IHR · HANDS-ON

Notebook 04

Evaluation: den Fortschritt messbar machen

- Alle fünf Ausbaustufen gegen 15 Gold-Fragen benchmarken
- Hit@3 und MRR pro Stufe vergleichen (mit Balkendiagramm)
- Sehen, warum erst der Metadaten-Filter Frage 13 löst

TEIL 3

Über RAG hinaus

Wann RAG nicht das richtige Werkzeug ist und was dann kommt

- Wissensproblem vs. Verhaltensproblem: RAG oder Fine-Tuning?
- LoRA und QLoRA: Fine-Tuning für alle
- Die Grenzen von statischem RAG
- Das ReAct-Paradigma und Agentic RAG



RAG oder Fine-Tuning?

„RAG oder Fine-Tuning?“ ist falsch gestellt. Die richtige Frage lautet: Was ist eigentlich das Problem?



Wissensproblem

Das Modell weiß etwas nicht → zu neu, zu spezifisch, nie im Training gesehen.

→ Lösung: RAG

Gibt dem Modell Information von außen.



Verhaltensproblem

Das Modell weiß es prinzipiell, antwortet aber nicht auf die benötigte Art.

→ Lösung: Fine-Tuning

Immer JSON · Unternehmensstil · Klausel-Extraktion · kleines Modell spezialisieren.

Kein Entweder-Oder: RAG gibt Information, Fine-Tuning verändert das Modell. In der Praxis werden beide oft kombiniert.

LoRA & QLoRA: Fine-Tuning für alle



LoRA · Low-Rank Adaptation

Statt alle Gewichte zu trainieren: Die Änderung ΔW wird als Produkt zweier kleiner Matrizen dargestellt ($\Delta W = B \cdot A$). Die Originalgewichte bleiben eingefroren.

Nur wenige Prozent trainierbar



QLoRA · noch effizienter

Kombiniert 4-Bit-Quantisierung des Basismodells mit LoRA-Adaptern in voller Präzision.

Llama-2-70B: statt ~140 GB VRAM reicht eine einzige 80-GB-GPU. Nur 0,04 % der Parameter trainierbar.



RAG + Fine-Tuning kombinieren: Ein juristischer Assistent nutzt Fine-Tuning für Stil und Terminologie, RAG für aktuelle Urteile. Oft besonders wirkungsvoll: das Embedding-Modell selbst auf Domänen-Paare fine-tunen.

Die Grenzen von statischem RAG



RAG ist einschrittig

Einmal retrieven, einmal generieren, fertig. Das reicht nicht für:

„Analysiere die Quartalsberichte der letzten drei Jahre, vergleiche sie mit unseren drei größten Wettbewerbern und fasse die strategischen Divergenzen zusammen.“

Ein Workflow statt eines einzelnen Schritts: retrieven, nachdenken, planen, integrieren, synthetisieren.

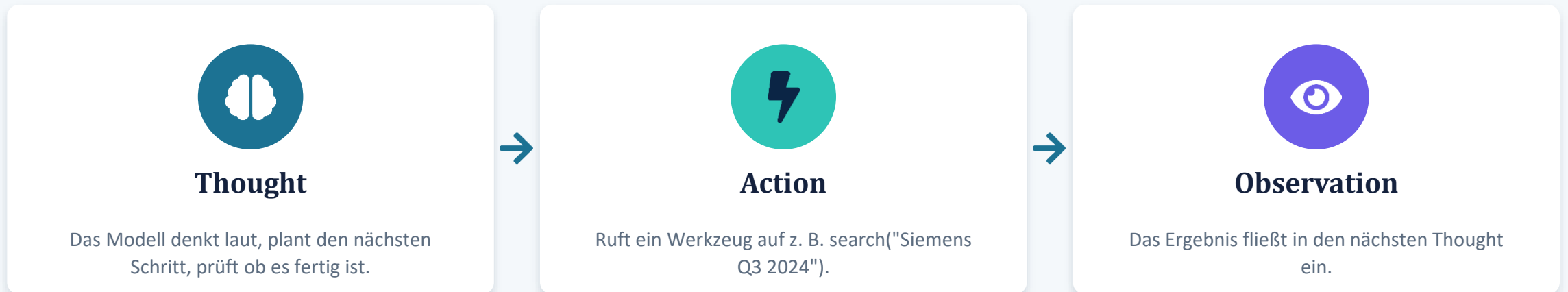
Was RAG nicht kann

- Multi-Step Reasoning über mehrere Quellen
- Dynamische Planung des nächsten Schritts
- Iteratives Nachsuchen mit neuen Begriffen
- Aufgaben mit eigener Abbruchbedingung

Genau hier setzt Agentic AI an.

Das ReAct-Paradigma

Reasoning + Acting (Yao et al., ICLR 2023): Nachdenken und Handeln wechseln sich ab, bis die Antwort steht.



↻ *Die Schleife wiederholt sich bis zur finalen Antwort*

Agentic RAG (Stand 2026)

Der Agent entscheidet dynamisch: Brauche ich Retrieval? Muss ich die Frage zerlegen? Reichen die Dokumente? Nochmal suchen?

Neue Risiken

Halluzinierte Werkzeugaufrufe · Endlosschleifen · Prompt Injection · Kosten & Latenz. Produktive Systeme brauchen Maximalschritte und Timeouts.

Chancen, Risiken, Enterprise-Tauglichkeit



Chancen

- Schnellster ROI: interne Wissenssuche
- Support-Entlastung & Onboarding
- Belegbare Antworten mit Quellen



Risiken

- Treue Antwort aus falscher Quelle (Air Canada)
- Veraltetes Wissen ohne Governance
- Blindes Vertrauen in flüssige Sprache



Enterprise-Reife

- Skalierbare Vektor-DB (pgvector)
- Zugriffsrechte & Metadaten-Pflege
- Monitoring + CI-Regressionstests

ZUM MITNEHMEN

Relevanz ist nicht Gültigkeit.



Naives RAG scheitert vorhersehbar, Hybrid Search + Reranking heben die Retrieval-Qualität deutlich.



Aber keine Retrieval-Stufe ersetzt gepflegte Metadaten und eine ehrliche Evaluation.



Erst messen, dann optimieren: ohne Gold-Testset ist jede Architekturdiskussion Geschmackssache.

Vielen Dank | Fragen?

Jannik Braunshausen · Theodor Höfer · Darnell Himmighöfer

WDSKI23A · DHBW Mannheim